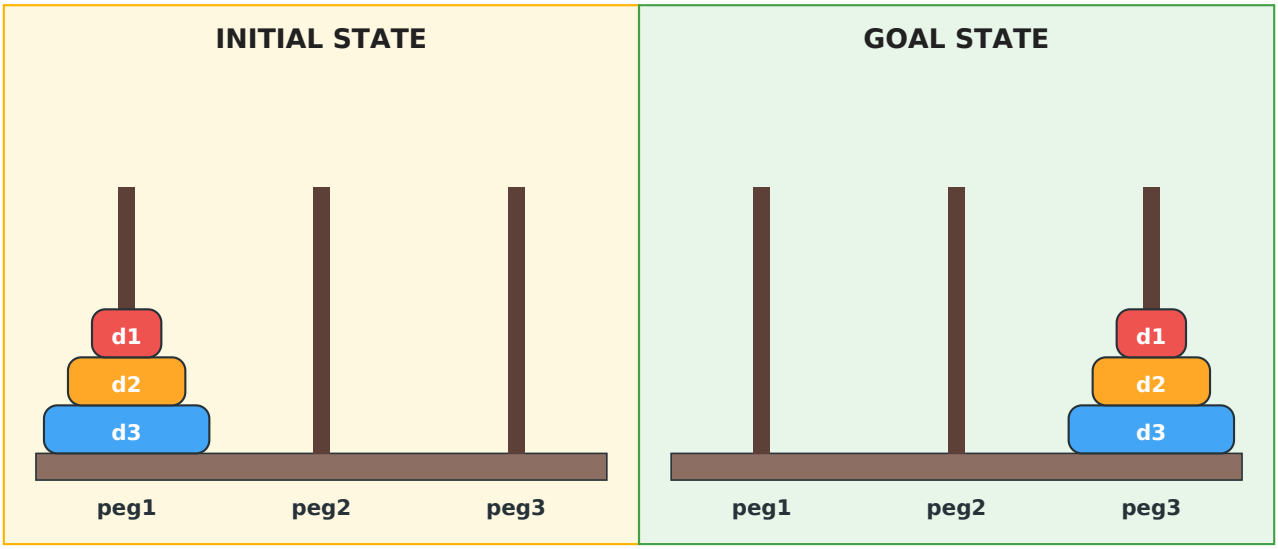
Problem 6: Tower of Hanoi

Level: **Intermediate** · Folder: **06_Tower_of_Hanoi/** · Files: domain.pddl, problem.pddl, solution_plan.txt

1. The problem

Three discs of different sizes (d1 smallest, d3 largest) are stacked on **peg1**. Move the whole stack to **peg3**. Only the top disc of a stack may be moved, one disc at a time, and a disc may **never** be placed on a smaller disc.



The goal picture shows one state that satisfies the goal (the final state of the plan). Only the facts in the :goal section are required; anything else may be different.

What you will learn

- A **type hierarchy**: peg and `disc` are both subtypes of `place`.
- Writing a game rule (size order) as **static facts** (*smaller*).
- One general action instead of many special cases.
- Exponential plan length: the optimum is 2^n - 1 moves.

2. Formulating the problem in PDDL

Step 1: Types and objects. peg: peg1, peg2, peg3. disc: d1, d2, d3. Both are *places* on which a disc can sit.

Step 2: Predicates describe what can be true or false in a state:

Predicate	Meaning
`(on ?d ?p)`	Disc ?d lies directly on ?p (a peg or another disc).
`(clear ?p)`	Nothing lies on ?p.
`(smaller ?d ?p)`	Disc ?d may be placed on ?p (static). True for every peg and every larger disc.

Step 3: Actions. Each action has *parameters*, a *precondition* (what must be true to apply it) and an *effect* (facts that become **true (+)** or **false (−)**):

Action	Precondition	Effect

move
`?d - disc, ?from - place, ?to - place`
Move clear disc ?d from ?from onto clear place ?to, if ?d is smaller than ?to.
	`(on ?d ?from)`
	`(clear ?d)`
	`(clear ?to)`
	`(smaller ?d ?to)`
		`+ (on ?d ?to)`
		`+ (clear ?from)`
		`− (on ?d ?from)`
		`− (clear ?to)`

Step 4: Initial state (`:init`) lists *every* fact that is true at the start. Everything else is false (closed-world assumption). **Step 5: Goal** (`:goal`) lists the facts that must be true at the end.

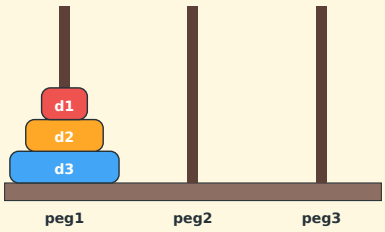
INITIAL STATE (18 facts)		GOAL
`(on d3 peg1)`	`(smaller d2 peg1)`	`(on d3 peg3)`
`(on d2 d3)`	`(smaller d2 peg2)`	`(on d2 d3)`
`(on d1 d2)`	`(smaller d2 peg3)`	`(on d1 d2)`
`(clear d1)`	`(smaller d3 peg1)`	
`(clear peg2)`	`(smaller d3 peg2)`	
`(clear peg3)`	`(smaller d3 peg3)`	
`(smaller d1 peg1)`	`(smaller d1 d2)`	
`(smaller d1 peg2)`	`(smaller d1 d3)`	
`(smaller d1 peg3)`	`(smaller d2 d3)`	

Modelling notes

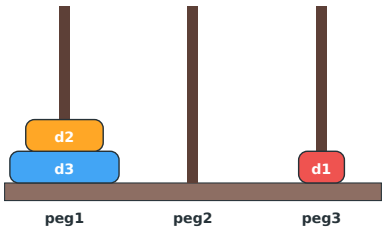
- Because ?from and ?to have type **place**, a single **move** action covers disc-to-peg, peg-to-disc, disc-to-disc and peg-to-peg moves.
- `(smaller d1 peg2)` may look strange. It just means 'a disc fits on an empty peg'.
- Type declarations must not be circular. Writing `place peg disc - place` on one line would make *place* a subtype of itself, so we write `place - object` separately.

3. Solution plan (7 actions, optimal)

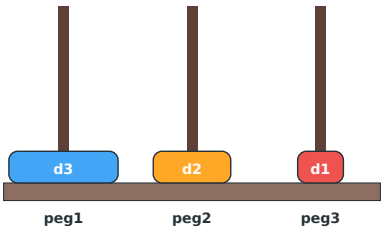
The planner finds the well-known 7-move solution: the smallest disc moves every second step. The pictures below show the state **after** each action. They were produced by executing the plan on the PDDL model.



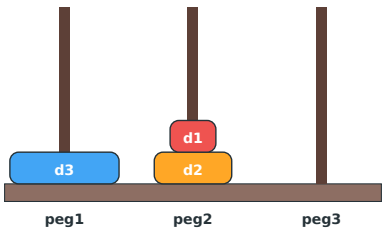
Initial state



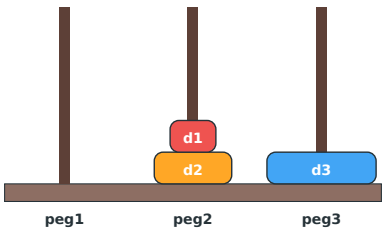
Step 1
`(move d1 d2 peg3)`



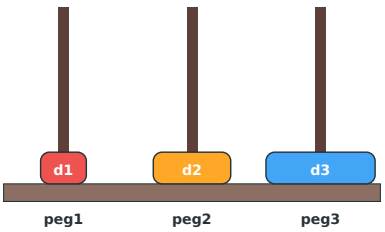
Step 2
`(move d2 d3 peg2)`



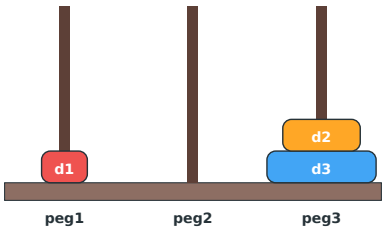
Step 3
`(move d1 peg3 d2)`



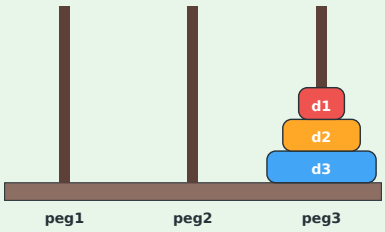
Step 4
`(move d3 peg1 peg3)`



Step 5
`(move d1 d2 peg1)`



Step 6
`(move d2 peg2 d3)`



Step 7
`(move d1 peg1 d2)`

4. How to run it

Online (no installation)	Open **editor.planning.domains**, paste *domain.pddl* and *problem.pddl* into two tabs and press **Solve**.
pyperplan (Python)	`pip install pyperplan` `pyperplan domain.pddl problem.pddl` (breadth-first search) `pyperplan -s astar -H lmcut domain.pddl problem.pddl` (optimal A*)
Fast Downward	`fast-downward.py domain.pddl problem.pddl --search "astar(lmcut())"`

5. Lab exercises

1. Solve with 4 discs (optimal: 15 moves). How many **smaller** facts do you need? (In general: 3n + n(n-1)/2.)
2. Change the goal to peg2.
3. Time the planner for 3, 4, 5 and 6 discs. What do you notice?

6. PDDL source code (appendix)

domain.pddl

```
;; ============================================================
;; Problem 6 : Tower of Hanoi
;; Level     : Intermediate
;; Move a stack of discs from the first peg to the third peg.
;; Only one disc moves at a time, and a larger disc may never
;; be placed on a smaller disc.
;; ============================================================
(define (domain hanoi)
  (:requirements :strips :typing)

  ;; Both pegs and discs are "places" a disc can sit on
  (:types place     - object
          peg disc  - place)

  (:predicates
    (on ?d - disc ?p - place)        ; disc ?d sits directly on ?p
    (clear ?p - place)               ; nothing is on top of ?p
    (smaller ?d - disc ?p - place)   ; disc ?d may be put on ?p
  )

  (:action move
    :parameters (?d - disc ?from - place ?to - place)
    :precondition (and (on ?d ?from)
                       (clear ?d)
                       (clear ?to)
                       (smaller ?d ?to))
    :effect (and (on ?d ?to)
                 (clear ?from)
                 (not (on ?d ?from))
                 (not (clear ?to)))
  )
)
```

problem.pddl

```
;; Three discs (d1 smallest, d3 largest) on peg1.
;; Goal: same stack on peg3.
(define (problem hanoi-3)
  (:domain hanoi)

  (:objects
    peg1 peg2 peg3 - peg
    d1 d2 d3       - disc
  )

  (:init
    ;; the stack on peg1:  d1 on d2 on d3 on peg1
```

```
    (on d3 peg1) (on d2 d3) (on d1 d2)
    (clear d1) (clear peg2) (clear peg3)

    ;; size relation: every disc fits on any peg ...
    (smaller d1 peg1) (smaller d1 peg2) (smaller d1 peg3)
    (smaller d2 peg1) (smaller d2 peg2) (smaller d2 peg3)
    (smaller d3 peg1) (smaller d3 peg2) (smaller d3 peg3)
    ;; ... and on any larger disc
    (smaller d1 d2) (smaller d1 d3)
    (smaller d2 d3)
  )

  (:goal (and (on d3 peg3) (on d2 d3) (on d1 d2)))
)
```

solution_plan.txt

```
1. (move d1 d2 peg3)
2. (move d2 d3 peg2)
3. (move d1 peg3 d2)
4. (move d3 peg1 peg3)
5. (move d1 d2 peg1)
6. (move d2 peg2 d3)
7. (move d1 peg1 d2)
```